

LAPORAN PRAKTIKUM

STUKTUR DATA

Algoritma Sorting (Insertion Sort, Selection Sort, dan Bubble Sort)

Menggunakan GUI

Disusun oleh:

Thaariq Salam

2511532022

Dosen Pengampu: Dr. Wahyudi S.T.M.T

Asisten Dosen: Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji dan syukur ke hadirat Allah SWT atas segala rahmat dan karunia-Nya, sehingga praktikan dapat menyelesaikan tugas dan penyusunan laporan Praktikum Struktur Data Pekan 7 yang berjudul "Algoritma Sorting (Insertion Sort, Selection Sort, dan Bubble Sort) Menggunakan GUI" dengan baik dan tepat waktu.

Laporan praktikum ini disusun sebagai salah satu syarat akademis dalam memenuhi komponen penilaian mata kuliah Praktikum Struktur Data pada Program Studi Informatika, Fakultas Teknologi Informasi, Universitas Andalas. Melalui penugasan pekan ini, praktikan telah berhasil mengimplementasikan struktur kelas ADT Mahasiswa, manipulasi objek di dalam ArrayList, pemanfaatan method `compareToIgnoreCase()` untuk komparasi string, serta visualisasi langkah demi langkah proses pengurutan (*sorting*) berbasis Java Swing GUI.

Saya menyadari bahwa keberhasilan penyusunan laporan ini tidak lepas dari bimbingan dan dukungan berbagai pihak. Oleh karena itu, saya menyampaikan terima kasih yang sebesar-besarnya kepada:

1. Bapak Dr. Wahyudi S.T.M.T selaku Dosen Pengampu mata kuliah Struktur Data.
2. Kak Sherly Sukmadira selaku Asisten Praktikum yang telah memberikan arahan selama pelaksanaan praktikum.

Saya menyadari sepenuhnya bahwa laporan praktikum ini masih jauh dari kata sempurna. Terdapat keterbatasan dalam analisis maupun teknis penulisan yang mungkin ditemukan. Oleh karena itu, saya sangat mengharapkan kritik yang membangun serta saran dari pembaca demi perbaikan di masa yang akan datang.

Padang, 21 Mei 2026

Thaariq Salam

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
1.3 Manfaat.....	2
BAB II PEMBAHASAN.....	3
2.1 Class InsertionSort_2511532022.java.....	3
2.1.1 Kode Program.....	3
2.1.2 Langkah Kerja.....	3
2.1.3 Analisis.....	4
2.2 Class SelectionSort_2511532022.java.....	5
2.2.1 Kode Program.....	5
2.2.2 Langkah Kerja.....	5
2.2.3 Analisis.....	6
2.3 Class BubbleSort_2511532022.java.....	7
2.3.1 Kode program.....	7
2.3.2 Langkah Kerja.....	7
2.3.3 Analisis.....	8
2.4 Class InsertionSortGUI_2511532022.java.....	9
2.4.1 Kode program.....	9
2.4.2 Langkah kerja dan Penjelasan Kode.....	12
2.4.3 Analisis.....	13
BAB III PENUTUP.....	14
1. Kode Program.....	15
1.1 Class Mahasiswa_2511532022.java (ADT).....	15
1.2 Class SortingMahasiswaGUI_2511532022.java.....	16
2. Langkah Kerja.....	19
3. Hasil Pengujian (<i>Output</i>).....	20
4. Kesimpulan.....	24

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengurutan data (*sorting*) merupakan salah satu operasi dasar dan paling krusial dalam bidang informatika dan ilmu komputer. Proses ini melibatkan penyusunan sekumpulan elemen data ke dalam urutan tertentu, baik secara menaik (*ascending*) maupun menurun (*descending*). Pengurutan data sangat penting karena struktur data yang teratur mempermudah pencarian elemen secara efisien menggunakan metode tertentu seperti *binary search*, mempercepat penggabungan data, serta meningkatkan kejelasan informasi visual.

Terdapat berbagai macam algoritma pengurutan dengan karakteristik, kompleksitas waktu, dan penggunaan memori yang berbeda. Di antaranya yang paling fundamental adalah *Bubble Sort*, *Selection Sort*, dan *Insertion Sort*. *Bubble Sort* bekerja dengan membandingkan pasangan elemen yang berdekatan secara berulang. *Selection Sort* memilih elemen terkecil atau terbesar lalu memindahkannya ke posisi yang tepat. Sementara itu, *Insertion Sort* menyisipkan sebuah elemen ke dalam sub-array yang sudah teratur pada tempat yang sesuai. Melalui praktikum pekan ke-7 ini, struktur dan alur kerja ketiga dasar metode pengurutan tersebut akan dipelajari dan diimplementasikan secara langsung menggunakan bahasa pemrograman Java, lengkap dengan visualisasi berbasis antarmuka grafis (GUI).

1.2 Tujuan

Tujuan dari pelaksanaan praktikum ini adalah sebagai berikut

1. Memahami konsep dasar, mekanisme operasional, dan logika dari algoritma *Bubble Sort*, *Selection Sort*, dan *Insertion Sort*.

2. Mengimplementasikan algoritma *Bubble Sort*, *Selection Sort*, dan *Insertion Sort* ke dalam bentuk kode program berbasis Java secara terstruktur.
3. Mampu melakukan analisis perbandingan performa dan pergeseran elemen data pada masing-masing algoritma pengurutan.
4. Mendesain antarmuka grafis (GUI) menggunakan pustaka Java Swing untuk memvisualisasikan langkah demi langkah (*step-by-step*) proses pengurutan data dengan metode *Insertion Sort*.

1.3 Manfaat

Manfaat yang didapat dari praktikum ini adalah sebagai berikut.

1. Mahasiswa dapat mengasah logika berpikir algoritmis dalam memecahkan masalah manipulasi array dan koleksi data.
2. Mahasiswa dapat memahami secara visual bagaimana suatu nilai berpindah tempat di dalam memori saat proses *sorting* berlangsung.
3. Memberikan fondasi pemahaman yang kuat bagi mahasiswa mengenai optimalisasi kode berdasarkan analisis kompleksitas algoritma untuk pengembangan sistem perangkat lunak yang lebih kompleks.

BAB II

PEMBAHASAN

2.1 Class InsertionSort_2511532022.java

Insertion Sort adalah algoritma pengurutan yang bekerja dengan cara memilah array menjadi dua bagian, yaitu bagian yang terurut dan bagian yang belum terurut. Algoritma ini mengambil elemen dari bagian yang belum terurut satu per satu, lalu menyisipkannya ke posisi yang tepat pada bagian yang sudah terurut, mirip seperti cara seseorang mengurutkan kartu di tangan.

2.1.1 Kode Program

```
1 package pekan7_2511532022;
2
3 public class InsertionSort_2511532022 {
4     public static void insertionSort(int[] arr) {
5         int n_2022 = arr.length;
6         for (int i_2022=1; i_2022 < n_2022; i_2022++) {
7             int key_2022 = arr[i_2022];
8             int j_2022 = i_2022 -1;
9             while (j_2022 >= 0 && arr[j_2022] > key_2022) {
10                 arr[j_2022+1] = arr[j_2022];
11                 j_2022--;
12             }
13             arr[j_2022+1] = key_2022;
14         }
15     }
16
17     public static void main (String[] args) {
18         int arr[] = {23, 78, 45, 8, 32, 56, 1};
19         int n_2022= arr.length;
20         System.out.printf("array yang belum terurut: \n");
21         for (int i_2022 = 0; i_2022 < n_2022; i_2022++) {
22             System.out.print(arr[i_2022] + " ");
23         }
24         System.out.println(" ");
25         insertionSort(arr);
26         System.out.printf("array yang terurut: \n");
27         for (int i_2022=0; i_2022 < n_2022; i_2022++) {
28             System.out.print(arr[i_2022] + " ");
29         }
30         System.out.println(" ");
31     }
32 }
```

2.1.2 Langkah Kerja

Berikut adalah penjelasan detail fungsi dari setiap bagian kode di atas.

- Perulangan luar `i_2022` berjalan maju dimulai dari posisi indeks ke-1 karena elemen pertama pada posisi indeks ke-0 secara default sudah dianggap berada di posisi terurut yang benar.
- Program mengambil nilai angka pada indeks aktif saat itu dan mengamankannya ke dalam variabel lokal `key_2022` sebagai nilai kunci yang akan disisipkan.
- Pointer mundur `j_2022` disiapkan tepat pada posisi satu indeks di belakang letak nilai kunci, yang diatur dengan cara indeks aktif dikurang 1 (`i_2022 - 1`).
- Perulangan `while` dijalankan mundur menyisir area kiri selama indeks pointer `j_2022` bernilai aman (lebih besar atau sama dengan 0) dan nilai elemen di kirinya terbukti lebih besar dari nilai kunci.
- Di dalam blok perulangan mundur tersebut, elemen yang berukuran lebih besar akan digeser ke arah kanan dengan cara menyalin datanya ke indeks yang ditambah 1 (`arr[j_2022+1] = arr[j_2022]`), kemudian nilai variabel `j_2022` dikurang 1 (`j_2022--`) agar melangkah mundur memeriksa data sebelumnya.
- Begitu perulangan mundur berhenti karena menemukan angka yang lebih kecil, nilai kunci `key_2022` langsung disisipkan secara pas ke dalam celah kosong yang ditinggalkan, yaitu pada lokasi indeks pointer ditambah 1 (`arr[j_2022+1]`).

2.1.3 Analisis

Insertion Sort bekerja mirip seperti mengurutkan kartu di tangan. Keunggulan utama algoritma ini adalah sifatnya yang adaptif. Jika data masukan dari awal sudah dalam kondisi terurut alami (*Best Case*), perulangan mundur `while` tidak akan pernah dieksekusi. Hal ini membuat performa kecepataannya melesat menjadi linear $O(n)$. Namun, jika data masukan berada dalam posisi terbalik (*Worst Case*), jumlah

pergeseran data (*shifting*) akan membesar secara kuadratik mencapai nilai kompleksitas $O(n^2)$.

```
<terminated> InsertionSort_2511532022 (
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

2.2 Class SelectionSort_2511532022.java

Selection Sort membagi susunan array menjadi dua bagian: sub-array bagian terurut di ujung kiri dan bagian yang belum terurut di kanan. Algoritma ini berulang kali mendeteksi elemen minimum dari bagian yang belum terurut lalu menukarnya dengan elemen pertama dari bagian belum terurut tersebut.

2.2.1 Kode Program

```
1 package pekan7_2511532022;
2
3 public class SelectionSort_2511532022 {
4     public static void selectionSort(int[] arr) {
5         int n_2022 = arr.length;
6         for (int i_2022 = 0; i_2022 < n_2022; i_2022++) {
7             int minIndex_2022 = i_2022;
8             for (int j_2022 = i_2022 + 1; j_2022 < n_2022; j_2022++) {
9                 if (arr[j_2022] < arr[minIndex_2022]) {
10                     minIndex_2022 = j_2022;
11                 }
12             }
13             int temp_2022 = arr[i_2022];
14             arr[i_2022] = arr[minIndex_2022];
15             arr[minIndex_2022] = temp_2022;
16         }
17     }
18     public static void main (String[] args) {
19         int arr[] = {23, 78, 45, 8, 32, 56, 1};
20         int n_2022 = arr.length;
21         System.out.printf("array yang belum terurut: \n");
22         for (int i_2022 = 0; i_2022 < n_2022; i_2022++) {
23             System.out.print(arr[i_2022] + " ");
24         }
25         System.out.println(" ");
26         selectionSort(arr);
27         System.out.printf("array yang terurut: \n");
28         for (int i_2022 = 0; i_2022 < n_2022; i_2022++) {
29             System.out.print(arr[i_2022] + " ");
30         }
31         System.out.println(" ");
32     }
33 }
```

2.2.2 Langkah Kerja

Berikut adalah penjelasan fungsi baris-baris kode pada class di atas

- Perulangan luar `i_2022` berjalan menyusuri array dari kiri ke kanan untuk menandai batas posisi mana yang saat ini sedang dicarikan angka terkecilnya.
- Program langsung mengasumsikan indeks awal putaran tersebut sebagai wadah angka paling kecil, lalu mencatat lokasinya ke variabel penunjuk `minIndex_2022`.
- Perulangan dalam `j_2022` mulai memindai sisa data di sebelah kanan, di mana titik mulainya diatur dengan cara indeks aktif ditambah 1 (`i_2022 + 1`) supaya program tidak sia-sia membandingkan suatu angka dengan dirinya sendiri.
- Kondisi pengecekan memeriksa data ke kanan, dan jika posisi indeks `j_2022` terbukti memegang angka yang lebih kecil dari angka di indeks minimum sementara, variabel `minIndex_2022` langsung diperbarui dengan nomor indeks baru tersebut.
- Setelah sisa array selesai dipindai penuh sampai ujung kanan, program keluar dari loop dalam dan menukar angka pada posisi target `i_2022` dengan angka paling kecil di posisi `minIndex_2022` secara sekaligus.

2.2.3 Analisis

Selection Sort memiliki keunggulan dalam hal efisiensi penukaran nilai data. Mekanisme pencarian elemen terkecil dilepaskan terlebih dahulu melalui perulangan dalam, dan eksekusi pertukaran (swap) hanya dipanggil tepat satu kali saja di akhir putaran perulangan luar. Jumlah operasi penukarannya bernilai konstan linear sebesar $O(n)$, namun kompleksitas untuk perbandingan datanya tetap berada pada nilai kuadratik $O(n^2)$ dalam kondisi data apa pun.

```
<terminated> SelectionSort_2511532022
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

2.3 Class BubbleSort_2511532022.java

Bubble Sort adalah algoritma pengurutan yang berulang kali menelusuri daftar array, membandingkan elemen yang berdekatan, dan menukarnya jika berada dalam urutan yang salah. Proses ini dianalogikan seperti gelembung udara yang bergerak naik ke permukaan.

2.3.1 Kode program

```
1 package pekan7_2511532022;
2 public class BubbleSort_2511532022 {
3     public static void bubbleSort(int[] arr) {
4         int n_2022= arr.length;
5         for (int i_2022 = 0; i_2022 < n_2022; i_2022++) {
6             for (int j_2022 = 0; j_2022 < n_2022- i_2022 -1; j_2022++) {
7                 if (arr[j_2022]>arr[j_2022+1]) {
8                     int temp_2022 = arr[j_2022];
9                     arr[j_2022] = arr[j_2022+1];
10                    arr[j_2022+1] = temp_2022;
11                    System.out.println("data: " + arr[j_2022] + " " + arr[j_2022+1]);
12                }
13            }
14        }
15    }
16    public static void main (String[]args) {
17        int arr[] = {23, 78, 45, 8, 32, 56, 1};
18        int n_2022= arr.length;
19        System.out.printf("array yang belum terurut: \n");
20        for (int i_2022 = 0; i_2022 < n_2022; i_2022++) {
21            System.out.print(arr[i_2022] + " ");
22        }
23        System.out.println(" ");
24        bubbleSort(arr);
25        System.out.printf("array yang terurut: \n");
26        for (int i_2022=0; i_2022 < n_2022; i_2022++) {
27            System.out.print(arr[i_2022] + " ");
28        }
29        System.out.println(" ");
30    }
31 }
```

2.3.2 Langkah Kerja

Berikut adalah penjelasan fungsi setiap baris kodenya.

- Program membaca total panjang data di dalam array dan menyimpannya ke variabel `n_2022`.
- Perulangan luar `i_2022` berjalan sebagai penghitung putaran, memastikan proses pengecekan diulang sebanyak jumlah elemen array agar seluruh data dipastikan terurut.

- Perulangan dalam `j_2022` bertugas menyisir indeks dari awal, dengan batas kanan yang sengaja dikurang `i_2022` dan dikurang 1 (`n_2022 - i_2022 - 1`) karena setiap akhir satu putaran besar, angka terbesar otomatis sudah aman "mengapung" di ujung paling kanan, sehingga area pencarian dipersempit agar program berjalan lebih efisien.
- Kondisi logika memeriksa apakah posisi angka saat ini `arr[j_2022]` nilainya lebih besar dari posisi tetangga tepat di depannya yang ditunjuk dengan cara ditambah 1 yaitu `arr[j_2022+1]`.
- Jika angka kiri terbukti lebih besar, nilainya diamankan ke variabel `temp_2022` agar tidak hilang, lalu nilai dari indeks ditambah 1 tadi digeser masuk ke kiri, dan posisi kanan diisi oleh nilai dari `temp_2022`.

2.3.3 Analisis

Algoritma *Bubble Sort* bekerja dengan menukar elemen yang bertetangga secara berulang-ulang jika urutannya salah. Pengurangan batas kanan berupa `n_2022 - i_2022 - 1` mencegah program melakukan perbandingan yang mubazir dengan data yang sudah pasti berada di posisi paling besar di bagian kanan array. Namun, karena algoritma ini selalu menggunakan perulangan bersarang untuk memeriksa seluruh elemen tanpa optimasi kondisi awal, kompleksitas waktu rata-rata dan terburuknya tetap sebesar $O(n^2)$.

```
array yang belum terurut:
23 78 45 8 32 56 1
data: 45 78
data: 8 78
data: 32 78
data: 56 78
data: 1 78
data: 8 45
data: 32 45
data: 1 56
data: 8 23
data: 1 45
data: 1 32
data: 1 23
data: 1 8
array yang terurut:
1 8 23 32 45 56 78
```

2.4 Class InsertionSortGUI_2511532022.java

Insertion Sort GUI adalah aplikasi antarmuka grafis interaktif berbasis Java Swing yang mengimplementasikan logika algoritma Insertion Sort untuk memvisualisasikan proses pergeseran angka di dalam memori secara langkah demi langkah melalui representasi objek kotak di layar.

2.4.1 Kode program

```
1 package pekan7_2511532022;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.FlowLayout;
7 import java.awt.Font;
8
9 import javax.swing.JFrame;
10 import javax.swing.JPanel;
11 import javax.swing.JScrollPane;
12 import javax.swing.JLabel;
13 import javax.swing.JOptionPane;
14 import javax.swing.BorderFactory;
15 import javax.swing.JButton;
16 import javax.swing.JTextField;
17 import javax.swing.SwingConstants;
18 import javax.swing.SwingUtilities;
19 import javax.swing.JTextArea;
20
21 public class InsertionSortGUI_2511532022 extends JFrame {
22
23     private static final long serialVersionUID = 1L;
24     private int[] array_2022;
25     private JLabel[] labelArray_2022;
26     private JButton stepButton_2022, resetButton_2022, setButton_2022;
27     private JTextField inputField_2022;
28     private JPanel panelArray_2022;
29     private JTextArea stepArea_2022;
30
31     private int i_2022 = 1, j_2022;
32     private boolean sorting_2022 = false;
33     private int stepCount_2022 = 1;
34
35     /**
36      * Launch the application.
37      */
38
39     /**
40      * Create the frame.
41      */
```

```

42 public InsertionSortGUI_2511532022() {
43     setTitle("Insertion Sort Langkah per Langkah");
44     setSize(800, 400);
45     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
46     setLocationRelativeTo(null);
47     setLayout(new BorderLayout());
48
49     //Panel input
50     JPanel inputPanel = new JPanel(new FlowLayout());
51     inputField_2022 = new JTextField(30);
52     setButton_2022 = new JButton("Set Array");
53     inputPanel.add(new JLabel("Masukkan angka (pisahkan dengan koma: ")");
54     inputPanel.add(inputField_2022);
55     inputPanel.add(setButton_2022);
56
57     //Panel array_2022 visual
58     panelArray_2022 = new JPanel();
59     panelArray_2022.setLayout(new FlowLayout());
60
61     //Panel kontrol
62     JPanel controlPanel = new JPanel();
63     stepButton_2022 = new JButton ("Langkah Selanjutnya");
64     resetButton_2022 = new JButton ("Reset");
65     stepButton_2022.setEnabled(false);
66     controlPanel.add(stepButton_2022);
67     controlPanel.add(resetButton_2022);
68
69     //Area teks untuk log langkah langkah
70     stepArea_2022 = new JTextArea(8, 60);
71     stepArea_2022.setEditable(false);
72     stepArea_2022.setFont(new Font("Monospaced", Font.PLAIN, 14));
73     JScrollPane scrollPane = new JScrollPane (stepArea_2022);
74
75     //Tambahkan panel ke frame
76     add(inputPanel, BorderLayout.NORTH);
77     add(panelArray_2022, BorderLayout.CENTER);
78     add(controlPanel, BorderLayout.SOUTH);
79     add(scrollPane, BorderLayout.EAST);
80
81
82 //Event Set Array
83 setButton_2022.addActionListener(e -> setArrayFromInput());
84
85 //Event Set Array
86 stepButton_2022.addActionListener(e -> performStep());
87
88 //Event Set Array
89 resetButton_2022.addActionListener(e -> reset());
90 }
91
92 public void setArrayFromInput() {
93     String text = inputField_2022.getText().trim();
94     if (text.isEmpty()) return;
95     String[] parts = text.split(",");
96     array_2022 = new int[parts.length];
97     try {
98         for (int k = 0; k < parts.length; k++) {
99             array_2022[k] = Integer.parseInt(parts[k].trim()); }
100     } catch (NumberFormatException e) {
101         JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan "
102             + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
103         return; }
104     i_2022 = 1;
105     stepCount_2022 = 1;
106     sorting_2022 = true;
107     stepButton_2022.setEnabled(true);
108     stepArea_2022.setText("");
109     panelArray_2022.removeAll();
110     labelArray_2022 = new JLabel(array_2022.length);
111     for (int k = 0; k < array_2022.length; k++) {
112         labelArray_2022[k] = new JLabel (String.valueOf(array_2022[k]));
113         labelArray_2022[k].setFont (new Font("Arial", Font.BOLD, 24));
114         labelArray_2022[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
115         labelArray_2022[k].setPreferredSize(new Dimension(50, 50));
116         labelArray_2022[k].setHorizontalAlignment(SwingConstants.CENTER);
117         panelArray_2022.add(labelArray_2022[k]);
118     }
119     panelArray_2022.revalidate();
120     panelArray_2022.repaint();
121 }

```

```

121
122 private void performStep() {
123     if (i_2022 < array_2022.length && sorting_2022) {
124         int key = array_2022[i_2022];
125         j_2022 = i_2022 - 1;
126
127         StringBuilder stepLog = new StringBuilder();
128         stepLog.append("Langkah ").append(stepCount_2022).
129         append(": Memasukkan ").append(key).append("\n");
130
131         while(j_2022 >= 0 && array_2022[j_2022] > key) {
132             array_2022[j_2022+1] = array_2022[j_2022];
133             j_2022--;
134         }
135         array_2022[j_2022+1] = key;
136
137         updateLabels();
138         stepLog.append("Hasil: ").append(arrayToString(array_2022)).append("\n\n");
139         stepArea_2022.append(stepLog.toString());
140
141         i_2022++;
142         stepCount_2022++;
143
144         if (i_2022== array_2022.length) {
145             sorting_2022 = false;
146             stepButton_2022.setEnabled(false);
147             JOptionPane.showMessageDialog(this, "Sorting selesai");
148         }
149     }
150 }
151
152 private void updateLabels() {
153     for(int k = 0; k < array_2022.length; k++) {
154         labelArray_2022[k].setText(String.valueOf(array_2022[k]));
155     }
156 }
157
158 private void reset() {
159     inputField_2022.setText("");
160     panelArray_2022.removeAll();
161     panelArray_2022.revalidate();
162     panelArray_2022.repaint();
163     stepArea_2022.setText("");
164     stepButton_2022.setEnabled(false);
165     sorting_2022 = false;
166     i_2022 = 1;
167     stepCount_2022 = 1;
168 }
169 private String arrayToString(int[] arr) {
170     StringBuilder sb = new StringBuilder();
171     for (int k = 0; k < array_2022.length; k++) {
172         sb.append(arr[k]);
173         if (k < array_2022.length - 1) sb.append(", ");
174     }
175     return sb.toString();
176 }
177 public static void main (String[]args) {
178     SwingUtilities.invokeLater(() -> {
179         InsertionSortGUI_2511532022 gui = new InsertionSortGUI_2511532022();
180         gui.setVisible(true);
181     });
182 }

```

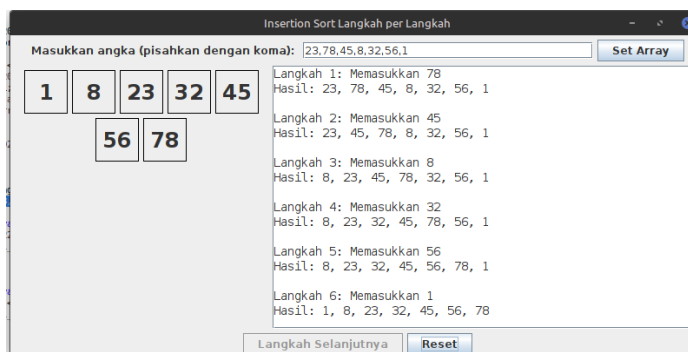
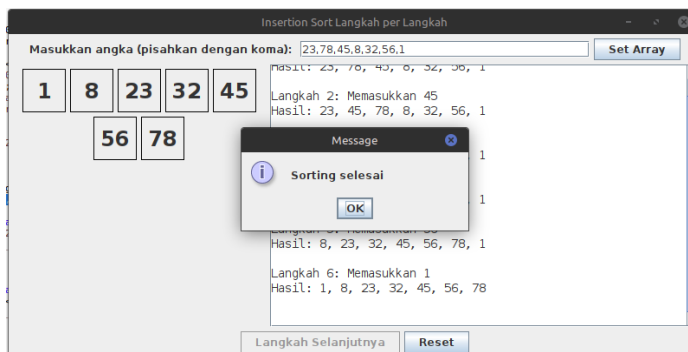
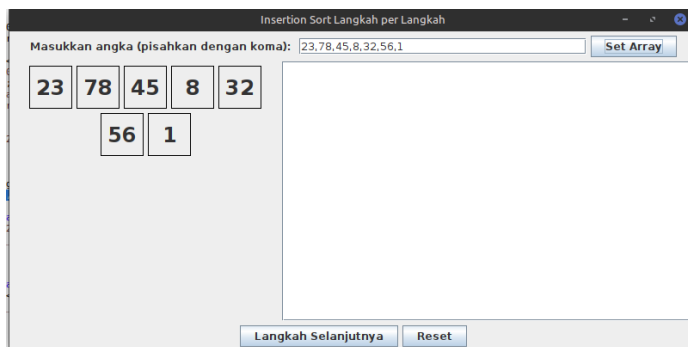
2.4.2 Langkah kerja dan Penjelasan Kode

Berikut adalah penjelasan fungsi setiap baris kodenya

- Konstruktor utama `InsertionSortGUI_2511532022` langsung menginisialisasi properti jendela frame, judul aplikasi, ukuran layar, posisi tengah, serta menetapkan aturan susunan letak wilayah menggunakan fungsi `BorderLayout`.
- Metode `setArrayFromInput()` bertugas menangkap rentetan teks masukan pengguna, memotong string tersebut berdasarkan simbol koma menggunakan fungsi `split()`, lalu mengubah tiap bagian teks menjadi tipe data integer di dalam proteksi blok try-catch.
- Setelah data sukses diurai, program menjalankan struktur perulangan untuk memproduksi objek komponen visual grafis `JLabel` secara dinamis dengan dimensi persegi berukuran 50x50 piksel, teks rata tengah, serta dibingkai garis border berwarna hitam.
- Fungsi `performStep()` bekerja saat pengguna mengklik tombol langkah selanjutnya untuk mengeksekusi tepat satu tahapan luar logika internal algoritma Insertion Sort dengan mengunci variabel `key` dan menjalankan perulangan `while` mundur untuk menggeser memori.
- Setelah data memori internal bergeser, fungsi `updateLabels()` dipanggil untuk memperbarui nilai teks angka pada kotak label visual layar komputer, menambah teks rekaman riwayat kemajuan langkah ke komponen `JTextArea`, serta menaikkan variabel hitungan indeks maju ditambah 1 (`i_2022++` dan `stepCount_2022++`) agar sistem siap mengeksekusi tahapan berikutnya.
- Kondisi `if` pada akhir langkah bertugas mendeteksi apabila nilai variabel perulangan maju sudah sama dengan panjang total elemen array, yang mana jika terpenuhi, program otomatis mematikan fungsi klik tombol kendali dan melempar kotak jendela pesan pop-up dialog dari objek `JOptionPane` untuk memberi tahu pengguna bahwa seluruh rangkaian pengurutan visual telah rampung secara utuh.

2.4.3 Analisis

Algoritma Insertion Sort versi GUI ini memfasilitasi pengguna dalam mempelajari penelusuran struktur data secara interaktif. Logika perulangan luar pada kode program pengurutan biasa yang aslinya berjalan otomatis beruntun tanpa jeda, berhasil dipisahkan dan diikat ke dalam event klik tombol langkah selanjutnya. Modifikasi ini mengubah sifat perulangan linier menjadi bentuk eksekusi manual per langkah (step-by-step) tanpa merusak nilai indeks internal memori (i_2022 dan j_2022), sehingga riwayat mutasi perpindahan posisi data dari setiap langkah pengerjaan dapat terekam dan ditampilkan dengan jelas pada area teks log.



BAB III

PENUTUP

Berdasarkan hasil pembuatan kode program, pengujian running, serta analisis yang telah dilakukan pada bab sebelumnya, maka dapat diambil beberapa kesimpulan sebagai berikut:

- Algoritma *Insertion Sort*, *Selection Sort*, dan *Bubble Sort* memiliki mekanisme pemindahan data yang berbeda di dalam memori array, di mana *Insertion Sort* mengandalkan penyisipan elemen ke area terurut, *Selection Sort* mengandalkan pencarian indeks nilai ekstrem terkecil/terbesar untuk ditukar di akhir putaran, dan *Bubble Sort* mengandalkan pertukaran konstan antara dua elemen yang saling bertetangga.
- Efisiensi waktu terbaik (*Best Case*) dengan kompleksitas linear $O(n)$ hanya dapat dicapai oleh algoritma *Insertion Sort* jika kondisi awal data di dalam array sudah berada dalam posisi terurut secara alami.
- Algoritma *Selection Sort* dan *Bubble Sort* memiliki karakteristik yang kurang adaptif terhadap kondisi awal data, sehingga dalam kondisi data acak maupun terbalik tetap dipaksa menjalankan perulangan bersarang (*nested loop*) yang menghasilkan kompleksitas waktu kuadratik $O(n^2)$.
- Implementasi algoritma ke dalam antarmuka grafis (*Graphical User Interface*) menggunakan *library* Java Swing berhasil memvisualisasikan jalan kerja *Insertion Sort* secara interaktif langkah demi langkah (*step-by-step*).
- Modifikasi pengurutan otomatis menjadi manual berbasis event klik tombol pada aplikasi GUI terbukti mempermudah pelacakan mutasi pergeseran indeks memori array tanpa merusak keaslian jalannya logika algoritma.

Pengembangan Tugas Praktikum Pekan 7: Struktur Data

Topik : Algoritma Sorting (Insertion Sort, Selection Sort, dan Bubble Sort)

Tema : Pengurutan Nama Mahasiswa Secara Alfabetis Menggunakan GUI

1. Kode Program

1.1 Class Mahasiswa_2511532022.java (ADT)

```
1 package pekan7_2511532022;
2 public class Mahasiswa_2511532022 {
3     private String nama_2022;
4     private String nim_2022;
5     private String prodi_2022;
6
7     public Mahasiswa_2511532022(String nama, String nim, String prodi) {
8         this.nama_2022 = nama;
9         this.nim_2022 = nim;
10        this.prodi_2022 = prodi;
11    }
12
13    public String getNama_2022() {
14        return nama_2022;
15    }
16
17    public void setNama_2022(String nama) {
18        this.nama_2022 = nama;
19    }
20
21    public String getNim_2022() {
22        return nim_2022;
23    }
24
25    public void setNim_2022(String nim) {
26        this.nim_2022 = nim;
27    }
28
29    public String getProdi_2022() {
30        return prodi_2022;
31    }
32
33    public void setProdi_2022(String prodi) {
34        this.prodi_2022 = prodi;
35    }
36
37    @Override
38    public String toString() {
39        return nama_2022 + " (" + nim_2022 + ")";
40    }
41 }
```

1.2 Class SortingMahasiswaGUI_2511532022.java

```
1 package pekan7_2511532022;
2
3 import java.awt.*;
4 import java.util.ArrayList;
5 import javax.swing.*;
6 import javax.swing.table.DefaultTableModel;
7
8 public class SortingMahasiswaGUI_2511532022 extends JFrame {
9     private static final long serialVersionUID = 1L;
10    private ArrayList<Mahasiswa_2511532022> listMahasiswa_2022;
11    private JTextField txtNama_2022, txtNim_2022, txtProdi_2022;
12    private JComboBox<String> cmbAlgoritma_2022;
13    private JTable tabelData_2022;
14    private DefaultTableModel tableModel_2022;
15    private JTextArea txtAreaLog_2022;
16    private JButton btnTambah_2022, btnHapus_2022, btnSort_2022;
17
18    public SortingMahasiswaGUI_2511532022() {
19        listMahasiswa_2022 = new ArrayList<>();
20        setTitle("Visualisasi Pengurutan Data Mahasiswa ");
21        setSize(950, 550);
22        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
23        setLocationRelativeTo(null);
24        setLayout(new BorderLayout(10, 10));
25
26        // Panel Input Form (Barat)
27        JPanel panelInput = new JPanel(new GridBagLayout());
28        panelInput.setBorder(BorderFactory.createTitledBorder("Form Input Mahasiswa"));
29        GridBagConstraints gbc = new GridBagConstraints();
30        gbc.insets = new Insets(5, 5, 5, 5);
31        gbc.fill = GridBagConstraints.HORIZONTAL;
32
33        gbc.gridx = 0; gbc.gridy = 0; panelInput.add(new JLabel("Nama:"), gbc);
34        gbc.gridx = 1; txtNama_2022 = new JTextField(15); panelInput.add(txtNama_2022, gbc);
35
36        gbc.gridx = 0; gbc.gridy = 1; panelInput.add(new JLabel("NIM:"), gbc);
37        gbc.gridx = 1; txtNim_2022 = new JTextField(15); panelInput.add(txtNim_2022, gbc);
38
39        gbc.gridx = 0; gbc.gridy = 2; panelInput.add(new JLabel("Prodi:"), gbc);
40        gbc.gridx = 1; txtProdi_2022 = new JTextField(15); panelInput.add(txtProdi_2022, gbc);
41    }
```

```

41
42 JPanel panelTombolForm = new JPanel(new FlowLayout());
43 btnTambah_2022 = new JButton("Tambah");
44 btnHapus_2022 = new JButton("Hapus");
45 panelTombolForm.add(btnTambah_2022);
46 panelTombolForm.add(btnHapus_2022);
47 gbc.gridx = 0; gbc.gridy = 3; gbc.gridwidth = 2; panelInput.add(panelTombolForm, gbc);
48
49 // Panel Kontrol Atas (Utara)
50 JPanel panelKontrol = new JPanel(new FlowLayout(FlowLayout.LEFT));
51 panelKontrol.setBorder(BorderFactory.createEtchedBorder());
52 cmbAlgoritma_2022 = new JComboBox<>(new String[]{"Insertion Sort", "Selection Sort", "Bubble Sort"});
53 btnSort_2022 = new JButton("Mulai Sorting");
54 panelKontrol.add(new JLabel("Pilih Algoritma: "));
55 panelKontrol.add(cmbAlgoritma_2022);
56 panelKontrol.add(btnSort_2022);
57
58 // Panel Data Tengah (Pusat) - JTable & Log Visualisasi
59 JPanel panelTengah = new JPanel(new GridLayout(2, 1, 5, 5));
60 String[] kolom = {"Nama Mahasiswa", "NIM", "Program Studi"};
61 tableModel_2022 = new DefaultTableModel(kolom, 0);
62 tabelData_2022 = new JTable(tableModel_2022);
63 JScrollPane scrollTabel = new JScrollPane(tabelData_2022);
64 scrollTabel.setBorder(BorderFactory.createTitledBorder("Daftar Tabel Mahasiswa"));
65
66 txtAreaLog_2022 = new JTextArea();
67 txtAreaLog_2022.setEditable(false);
68 txtAreaLog_2022.setFont(new Font("Monospaced", Font.PLAIN, 13));
69 JScrollPane scrollLog = new JScrollPane(txtAreaLog_2022);
70 scrollLog.setBorder(BorderFactory.createTitledBorder("Log Langkah-Langkah Visualisasi Sorting"));
71
72 panelTengah.add(scrollTabel);
73 panelTengah.add(scrollLog);
74
75 // Satukan ke Frame
76 add(panelInput, BorderLayout.WEST);
77 add(panelKontrol, BorderLayout.NORTH);
78 add(panelTengah, BorderLayout.CENTER);
79
80 // Event Listener Tombol
81 btnTambah_2022.addActionListener(e -> tambahData());
82 btnHapus_2022.addActionListener(e -> hapusData());
83 btnSort_2022.addActionListener(e -> eksekusiSorting());
84 }
85
86 private void tambahData() {
87     String nama = txtNama_2022.getText().trim();
88     String nim = txtNim_2022.getText().trim();
89     String prodi = txtProdi_2022.getText().trim();
90
91     if (nama.isEmpty() || nim.isEmpty() || prodi.isEmpty()) {
92         JOptionPane.showMessageDialog(this, "Semua input form wajib diisi!", "Peringatan", JOptionPane.WARNING_MESSAGE);
93         return;
94     }
95
96     Mahasiswa_2511532022 mhs = new Mahasiswa_2511532022(nama, nim, prodi);
97     listMahasiswa_2022.add(mhs);
98     tableModel_2022.addRow(new Object[]{mhs.getNama_2022(), mhs.getNim_2022(), mhs.getProdi_2022()});
99
100     txtNama_2022.setText("");
101     txtNim_2022.setText("");
102     txtProdi_2022.setText("");
103 }
104
105 private void hapusData() {
106     int barisTerpilih = tabelData_2022.getSelectedRow();
107     if (barisTerpilih == -1) {
108         JOptionPane.showMessageDialog(this, "Pilih salah satu baris tabel yang ingin dihapus!", "Peringatan", JOptionPane.WARNING_MESSAGE);
109         return;
110     }
111     listMahasiswa_2022.remove(barisTerpilih);
112     tableModel_2022.removeRow(barisTerpilih);
113 }
114
115 private void updateTabelDariArrayList() {
116     tableModel_2022.setRowCount(0);
117     for (Mahasiswa_2511532022 mhs : listMahasiswa_2022) {
118         tableModel_2022.addRow(new Object[]{mhs.getNama_2022(), mhs.getNim_2022(), mhs.getProdi_2022()});
119     }
120 }

```

```

121
122 private void eksekusiSorting() {
123     if (listMahasiswa_2022.size() < 2) {
124         JOptionPane.showMessageDialog(this, "Data mahasiswa minimal harus berjumlah 2 untuk diurutkan!", "Info", JOptionPane.INFORMATION_MESSAGE);
125         return;
126     }
127
128     txtAreaLog_2022.setText("");
129     String opsi = (String) cmbAlgoritma_2022.getSelectedItem();
130
131     if (opsi.equals("Insertion Sort")) {
132         insertionSort();
133     } else if (opsi.equals("Selection Sort")) {
134         selectionSort();
135     } else if (opsi.equals("Bubble Sort")) {
136         bubbleSort();
137     }
138
139     updateTabelDariArrayList();
140 }
141
142 private void insertionSort() {
143     txtAreaLog_2022.append("=== INTI LOGIKA INSERTION SORT ===\n");
144     txtAreaLog_2022.append("Data Awal: " + listMahasiswa_2022.toString() + "\n\n");
145     int n = listMahasiswa_2022.size();
146     int langkah = 1;
147
148     for (int i_2022 = 1; i_2022 < n; i_2022++) {
149         Mahasiswa_2511532022 key_2022 = listMahasiswa_2022.get(i_2022);
150         int j_2022 = i_2022 - 1;
151
152         while (j_2022 >= 0 && listMahasiswa_2022.get(j_2022).getNama_2022().compareToIgnoreCase(key_2022.getNama_2022()) > 0) {
153             listMahasiswa_2022.set(j_2022 + 1, listMahasiswa_2022.get(j_2022));
154             j_2022--;
155         }
156         listMahasiswa_2022.set(j_2022 + 1, key_2022);
157         txtAreaLog_2022.append("Langkah " + (langkah++) + " : " + listMahasiswa_2022.toString() + "\n");
158     }
159     txtAreaLog_2022.append("\nHasil Akhir: " + listMahasiswa_2022.toString() + "\n");
160 }
161
162 private void selectionSort() {
163     txtAreaLog_2022.append("=== INTI LOGIKA SELECTION SORT ===\n");
164     txtAreaLog_2022.append("Data Awal: " + listMahasiswa_2022.toString() + "\n\n");
165     int n = listMahasiswa_2022.size();
166     int pass = 1;
167     for (int i_2022 = 0; i_2022 < n - 1; i_2022++) {
168         int minIndex_2022 = i_2022;
169         for (int j_2022 = i_2022 + 1; j_2022 < n; j_2022++) {
170             if (listMahasiswa_2022.get(j_2022).getNama_2022().compareToIgnoreCase(listMahasiswa_2022.get(minIndex_2022).getNama_2022()) < 0) {
171                 minIndex_2022 = j_2022;
172             }
173         }
174         Mahasiswa_2511532022 temp_2022 = listMahasiswa_2022.get(i_2022);
175         listMahasiswa_2022.set(i_2022, listMahasiswa_2022.get(minIndex_2022));
176         listMahasiswa_2022.set(minIndex_2022, temp_2022);
177         txtAreaLog_2022.append("Pass " + (pass++) + " : " + listMahasiswa_2022.toString() + "\n");
178     }
179     txtAreaLog_2022.append("\nHasil Akhir: " + listMahasiswa_2022.toString() + "\n");
180 }
181
182 private void bubbleSort() {
183     txtAreaLog_2022.append("=== INTI LOGIKA BUBBLE SORT ===\n");
184     txtAreaLog_2022.append("Data Awal: " + listMahasiswa_2022.toString() + "\n\n");
185     int n = listMahasiswa_2022.size();
186     int pass = 1;
187
188     for (int i_2022 = 0; i_2022 < n; i_2022++) {
189         boolean adaPertukaran = false;
190         for (int j_2022 = 0; j_2022 < n - i_2022 - 1; j_2022++) {
191             if (listMahasiswa_2022.get(j_2022).getNama_2022().compareToIgnoreCase(listMahasiswa_2022.get(j_2022 + 1).getNama_2022()) > 0) {
192                 Mahasiswa_2511532022 temp_2022 = listMahasiswa_2022.get(j_2022);
193                 listMahasiswa_2022.set(j_2022, listMahasiswa_2022.get(j_2022 + 1));
194                 listMahasiswa_2022.set(j_2022 + 1, temp_2022);
195                 adaPertukaran = true;
196             }
197         }
198         txtAreaLog_2022.append("Pass " + (pass++) + " : " + listMahasiswa_2022.toString() + "\n");
199         if (!adaPertukaran) break;
200     }
201     txtAreaLog_2022.append("\nHasil Akhir: " + listMahasiswa_2022.toString() + "\n");
202 }
203
204 public static void main(String[] args) {
205     SwingUtilities.invokeLater(() -> {
206         new SortingMahasiswaGUI_2511532022().setVisible(true);
207     });
208 }
209 }

```

2. Langkah Kerja

Langkah kerja atau Alur program telah sesuai dengan apa yang diberitahu pada file penugasan:

4. Alur Program

1. Pengguna menginput data mahasiswa melalui GUI.
2. Data disimpan ke dalam array atau ArrayList.
3. Pengguna memilih algoritma sorting pada ComboBox.
4. Pengguna menekan tombol "Mulai Sorting".
5. Program menjalankan algoritma sorting yang dipilih.
6. Proses sorting ditampilkan visualisasinya langkah demi langkah.
7. Hasil akhir pengurutan ditampilkan pada GUI.

Berikut adalah penjelasan detail fungsi dari setiap bagian kode di atas.

- Program mendefinisikan kelas ADT bernama Mahasiswa_2511532022 untuk membungkus variabel nama_2022, nim_2022, dan prodi_2022 menjadi satu kesatuan objek entitas tunggal yang utuh.
- Method *overriding toString()* ditambahkan pada kelas ADT agar saat objek mahasiswa dipanggil di dalam penulisan log visualisasi, teks yang keluar secara otomatis berformat nama dan nomor induk mahasiswa.
- Konstruktor utama SortingMahasiswaGUI_2511532022 menyiapkan alokasi penampung objek array dinamis ArrayList, tata letak layout form, pembuatan kolom JTable, pembuatan ComboBox pilihan algoritma, serta mengikat fungsi tombol tambah, hapus, dan urutkan.
- Fungsi tambahData() menyaring teks isian form JTextField agar tidak kosong, membungkus data tersebut ke dalam instansiasi objek kelas ADT baru, kemudian memasukkannya secara bersamaan ke dalam ArrayList internal program serta baris visual tabel data mahasiswa.
- Fungsi hapusData() bertugas membaca indeks baris horizontal tabel yang sedang diklik atau disorot oleh kursor pengguna untuk dibuang datanya secara sinkron dari ArrayList serta baris JTable.

- Fungsi `eksekusiSorting()` melakukan validasi kuantitas elemen data di dalam `ArrayList` agar minimal berjumlah dua objek, membersihkan isi teks komponen `JTextArea` log lama, membaca teks pilihan algoritma aktif pada `ComboBox`, lalu mengarahkan program menuju pemanggilan method `sorting` yang sesuai.
- Method `insertionSort()` mengunci satu referensi objek mahasiswa pada perulangan luar maju, lalu menggunakan struktur perulangan `while` mundur untuk memindahkan referensi objek ke sisi kanan selama nilai nama mahasiswa pada indeks di kirinya menghasilkan nilai integer lebih besar dari nol saat diuji via perbandingan `compareToIgnoreCase()`.
- Method `selectionSort()` bekerja menetapkan asumsi indeks penunjuk elemen string terkecil sementara, memindai seluruh sisa nama objek mahasiswa di sebelah kanan via perulangan bersarang bersyarat kurang dari nol, lalu menukar letak objek tersebut tepat satu kali di akhir perulangan luar.
- Method `bubbleSort()` menyisir dan membandingkan elemen nama objek mahasiswa yang saling bertetangga berdampingan dari indeks awal, lalu secara bertahap melakukan operasi `swap` pemindahan posisi objek ke kanan jika nama sebelah kiri berstatus alfabetis lebih besar dari objek sebelah kanannya.
- Rangkaian proses pencatatan visualisasi teks mutasi data mahasiswa di setiap akhir putaran internal disalin dan dilempar secara berurutan ke dalam text area log serta diakhiri dengan pemanggilan `updateTabelDariArrayList()` untuk meremajakan visual grafis tabel utama layar komputer.

3. Hasil Pengujian (*Output*)

Sistem pengurutan ini menggunakan `ArrayList` untuk memanipulasi referensi alamat objek kelas ADT secara dinamis di dalam memori tanpa perlu melakukan

instansiasi ulang isi data atribut mahasiswa. Evaluasi performa pengurutan teks alfabetis menggunakan method **compareToIgnoreCase()** tetap terikat pada standar kompleksitas asimtotik waktu algoritma asalnya, di mana Insertion Sort, Selection Sort, dan Bubble Sort secara umum memproduksi nilai rata-rata kuadratik $O(n^2)$.

Namun, modifikasi penggunaan fungsi bawaan string Java ini memberikan jaminan akurasi tinggi pada kestabilan pengurutan abjad (A-Z) terlepas dari adanya variasi pengetikan huruf kapital maupun kecil dari data yang dimasukkan oleh pengguna lewat antarmuka grafis JTable.

Screenshot Tampilan Utama Aplikasi GUI Beserta Isi Catatan Log Langkah Pilihan Pengurutan (*sorting*) :

Nama Mahasiswa	NIM	Program Studi
----------------	-----	---------------

Menginput data mahasiswa melalui GUI lalu data disimpan ke dalam array atau ArrayList

Visualisasi Pengurutan Data Mahasiswa

Pilih Algoritma: Insertion Sort

Mulai Sorting

Form Input Mahasiswa

Nama: Zaki

NIM: 2511533006

Prodi: Informatika

Tambah Hapus

Daftar Tabel Mahasiswa

Nama Mahasiswa	NIM	Program Studi
Thaariq	2511532022	Informatika
Faiz	2511533026	Informatika

Log Langkah-Langkah Visualisasi Sorting

Pilih menu pilihan sorting , lalu tekan tombol Mulai Sorting, hasil dan langkah kerja akan muncul di Log/ tampilan GUI.

Tugas Sorting Pekan 7.pdf

Visualisasi Pengurutan Data Mahasiswa

Pilih Algoritma: Bubble Sort

Mulai Sorting

Form Input Mahasiswa

Nama:

NIM:

Prodi:

Tambah Hapus

Daftar Tabel Mahasiswa

Nama Mahasiswa	NIM	Program Studi
Faiz	2511533026	Informatika
Thaariq	2511532022	Informatika
Zaki	2511533006	Informatika

Log Langkah-Langkah Visualisasi Sorting

```

===== INTI LOGIKA BUBBLE SORT =====
Data Awal: [Faiz (2511533026), Thaariq (2511532022), Zaki (2511533006)]
Pass 1 : [Faiz (2511533026), Thaariq (2511532022), Zaki (2511533006)]
Hasil Akhir: [Faiz (2511533026), Thaariq (2511532022), Zaki (2511533006)]

```

Insertion Sort

Visualisasi Pengurutan Data Mahasiswa

Pilih Algoritma: Insertion Sort

Mulai Sorting

Form Input Mahasiswa

Nama:

NIM:

Prodi:

Tambah Hapus

Daftar Tabel Mahasiswa

Nama Mahasiswa	NIM	Program Studi
Faiz	2511533026	Informatika
Thaariq	2511532022	Informatika
Zaki	2511533006	Informatika

Log Langkah-Langkah Visualisasi Sorting

```

===== INTI LOGIKA INSERTION SORT =====
Data Awal: [Thaariq (2511532022), Faiz (2511533026), Zaki (2511533006)]
Langkah 1 : [Faiz (2511533026), Thaariq (2511532022), Zaki (2511533006)]
Langkah 2 : [Faiz (2511533026), Thaariq (2511532022), Zaki (2511533006)]
Hasil Akhir: [Faiz (2511533026), Thaariq (2511532022), Zaki (2511533006)]

```

Selection sort

Visualisasi Pengurutan Data Mahasiswa

Pilih Algoritma: **Selection Sort** **Mulai Sorting**

Form Input Mahasiswa

Nama:

NIM:

Prodi:

Tambah **Hapus**

Daftar Tabel Mahasiswa

Nama Mahasiswa	NIM	Program Studi
Faiz	2511533026	Informatika
Thaariq	2511532022	Informatika
Zaki	2511533006	Informatika

Log Langkah-Langkah Visualisasi Sorting

=== INTI LOGIKA SELECTION SORT ===

Data Awal: [Faiz (2511533026), Thaariq (2511532022), Zaki (2511533006)]

Pass 1 : [Faiz (2511533026), Thaariq (2511532022), Zaki (2511533006)]

Pass 2 : [Faiz (2511533026), Thaariq (2511532022), Zaki (2511533006)]

Hasil Akhir: [Faiz (2511533026), Thaariq (2511532022), Zaki (2511533006)]

Bubble Sort

Visualisasi Pengurutan Data Mahasiswa

Pilih Algoritma: **Bubble Sort** **Mulai Sorting**

Form Input Mahasiswa

Nama:

NIM:

Prodi:

Tambah **Hapus**

Daftar Tabel Mahasiswa

Nama Mahasiswa	NIM	Program Studi
Faiz	2511533026	Informatika
Thaariq	2511532022	Informatika
Zaki	2511533006	Informatika

Log Langkah-Langkah Visualisasi Sorting

=== INTI LOGIKA BUBBLE SORT ===

Data Awal: [Faiz (2511533026), Thaariq (2511532022), Zaki (2511533006)]

Pass 1 : [Faiz (2511533026), Thaariq (2511532022), Zaki (2511533006)]

Hasil Akhir: [Faiz (2511533026), Thaariq (2511532022), Zaki (2511533006)]

Tekan tombol hapus untuk menghapus list atau elemen dalam arraylist

Visualisasi Pengurutan Data Mahasiswa

Pilih Algoritma: **Bubble Sort** **Mulai Sorting**

Form Input Mahasiswa

Nama:

NIM:

Prodi:

Tambah **Hapus**

Daftar Tabel Mahasiswa

Nama Mahasiswa	NIM	Program Studi
Thaariq	2511532022	Informatika
Zaki	2511533006	Informatika

Log Langkah-Langkah Visualisasi Sorting

=== INTI LOGIKA BUBBLE SORT ===

Data Awal: [Faiz (2511533026), Thaariq (2511532022), Zaki (2511533006)]

Pass 1 : [Faiz (2511533026), Thaariq (2511532022), Zaki (2511533006)]

Hasil Akhir: [Faiz (2511533026), Thaariq (2511532022), Zaki (2511533006)]

4. Kesimpulan

Berdasarkan hasil pembuatan kode program, pengujian running, serta analisis yang telah dilakukan pada bab sebelumnya, maka dapat diambil beberapa kesimpulan sebagai berikut:

- Pengurutan data berbasis objek teks *String* pada bahasa pemrograman Java wajib diselesaikan menggunakan perbandingan leksikografis method `compareToIgnoreCase()` agar tidak membedakan jenis huruf kapital dan kecil.
- Struktur data dinamis `ArrayList` terbukti sangat adaptif untuk digunakan bersama kelas *Abstract Data Type* (ADT) karena memudahkan operasi penambahan, penghapusan, serta manipulasi indeks elemen saat terjadi pertukaran posisi sorting mahasiswa.
- Komponen interaktif `JComboBox` pada pustaka komponen Java Swing dapat dikonfigurasi sebagai sakelar kontrol kendali untuk menentukan metode fungsi sorting mana yang akan dipanggil oleh sistem saat aksi penekanan tombol dipicu oleh pengguna.
- Visualisasi langkah demi langkah pengurutan objek mahasiswa yang ditampilkan langsung pada area komponen log `JTextArea` sangat membantu pengamatan perubahan posisi data secara bertahap sesuai alur komparasi algoritma aslinya.
- Integrasi project Eclipse dengan perkakas plugin `EGit` mempermudah pengelolaan berkas kode program serta menjamin keamanan rekam jejak revisi tugas praktikum secara terstruktur saat diunggah menuju `GitHub`.